

Heap Basics - Interview Notes

1. What is a Heap?

A Heap is a Complete Binary Tree that follows Heap Property.

Max Heap: Parent $\geq$ Children

Min Heap: Parent $\leq$ Children

2. Complete Binary Tree

All levels are completely filled except possibly the last level. The last level must be filled from left to right.

3. Heap Array Representation

Root is always stored at index 0.

Parent = $(i-1)/2$

Left Child = $2*i+1$

Right Child = $2*i+2$

4. Is Heap Sorted?

No. Heap only guarantees Parent vs Child relationship. Sibling nodes need not be sorted.

5. Important Time Complexities

Insert: $O(\log n)$

Delete Root: $O(\log n)$

Peek: $O(1)$

Build Heap: $O(n)$

6. Leaf and Non-Leaf Nodes

Leaf nodes start from index $n/2$ till $n-1$.

Last leaf node index = $n-1$.

Last non-leaf node index = $(n/2)-1$.

Why do we start from last non-leaf node?

Leaf nodes are already valid heaps. Only parent nodes need fixing. Therefore heap construction starts from $(n/2)-1$ and moves upward.

7. Heapify

Heapify = Compare + Swap + Continue recursively until heap property is restored.

8. Build Heap Algorithm

```
for(int i=(n/2)-1; i>=0; i--)
```

```
heapify(arr,n,i);
```

9. Example: Build Max Heap

Input: [4,10,3,5,1]

$n = 5$

Last non-leaf = $(5/2)-1 = 1$

Heapify index 1:

10 has children 5 and 1 -> no swap.

Heapify index 0:

4 has children 10 and 3 -> swap with 10.

Array becomes [10,4,3,5,1]

Heapify index 1:

4 has children 5 and 1 -> swap with 5.

Final Heap = [10,5,3,4,1]

10. Interview Questions & Answers

Q: Why start from $(n/2)-1$?

A: It is the last non-leaf node. Leaf nodes are already valid heaps.

Q: Is swapping called heapify?

A: No. Heapify includes comparison, swap and recursive fixing.

Q: Is root stored separately?

A: No. Root is always `arr[0]`.

Q: Can build-heap start from any index?

A: No. It must start from last non-leaf node and move upward.

Q: Difference between last leaf node and last non-leaf node?

A: Last leaf = $n-1$. Last non-leaf = $(n/2)-1$.

Next Topics

1. Insert into Heap (Heapify Up)
2. Delete Root (Heapify Down)
3. Java PriorityQueue
4. Kth Largest Element
5. Top K Frequent Elements